David King
Dr. Tong Zhang
Advanced Computer Systems
20 October 2026

Classwide Project: SIMD Advantage Profiling

Introduction

This report compares scalar and compiler-generated SIMD implementations of three C kernels using AVX instructions: AXPY (y = a*x + y), dot product, and element-wise multiplication. We examine how array size, float or double precision, alignment, and access pattern affect speedup and throughput.

System Configuration

Item	Configuration
CPU / ISA	Intel Core i7-13800H; AVX
Operating system	Ubuntu 24.04.5 LTS; Linux 6.8.0-147-generic
Compiler	GCC 13.3.0
Compiler flags	-O3 -mavx -ffast-math -ffp-contract=off; scalar additionally: -fno-tree-vectorize -fno-tree-slp-vectorize
Frequency policy	powersave governor
SMT state	Enabled

Measurement Procedure

1. Pin the benchmark's single thread to logical CPU 0.
2. Allocate and initialize the arrays before timing. For the main sweep, test 53 array sizes from 24 to 33,554,432 elements. Align arrays to 64-byte boundaries and choose sizes that fill complete vectors.

3. Run five untimed warmup calls before each measurement.
4. Measure only kernel execution time. Time one call per measurement in the main sweeps.
5. Repeat each case 15 times, alternating implementation order and shuffling the sweep order. Report medians and interquartile ranges. Validate outputs after timing.

Pre-experiment Expectations

We expected SIMD to help most when the input arrays fit in cache, with smaller gains once memory bandwidth became the main bottleneck. Because float values are smaller than double values, we expected to process more of them in the same amount of time, leading to greater speedup and throughput. Misalignment, leftover elements, and non-unit strides were expected to reduce performance.

Experiments and Results

Scalar Baseline and Auto-vectorized Performance

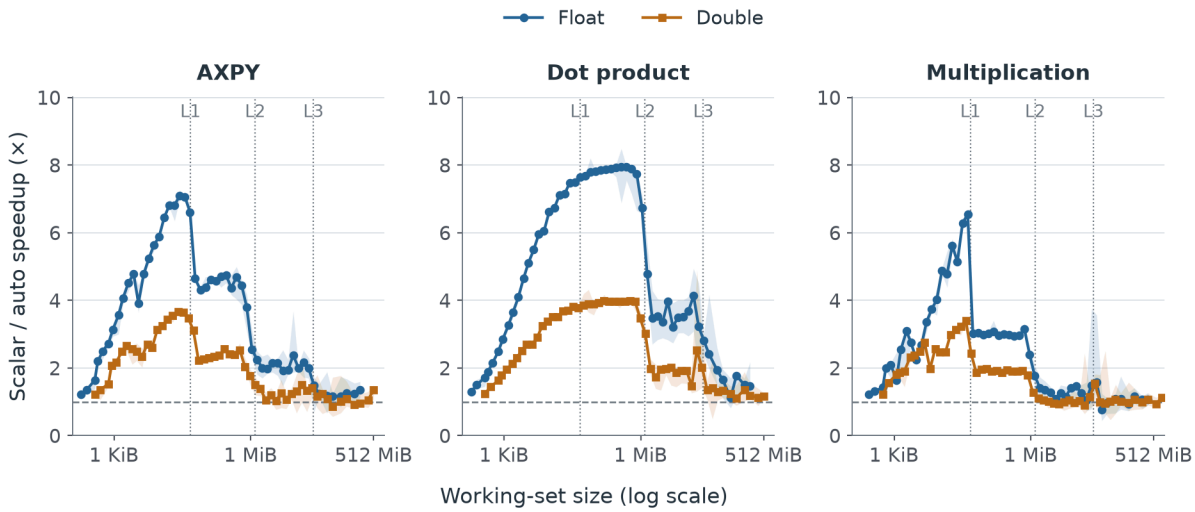


Fig. 1. SIMD speedup (scalar time / auto-vectorized time) across working-set sizes. Higher is faster; 1× means equal performance. Bands show the interquartile range, and cache markers indicate nominal capacities.

Peak median speedups were 7.10× for float AXPY, 7.95× for dot product, and 6.54× for multiplication (Figure 1). Double-precision peaks were 3.66×, 4.00×, and 3.41×. Float therefore achieved greater peak speedup for every kernel.

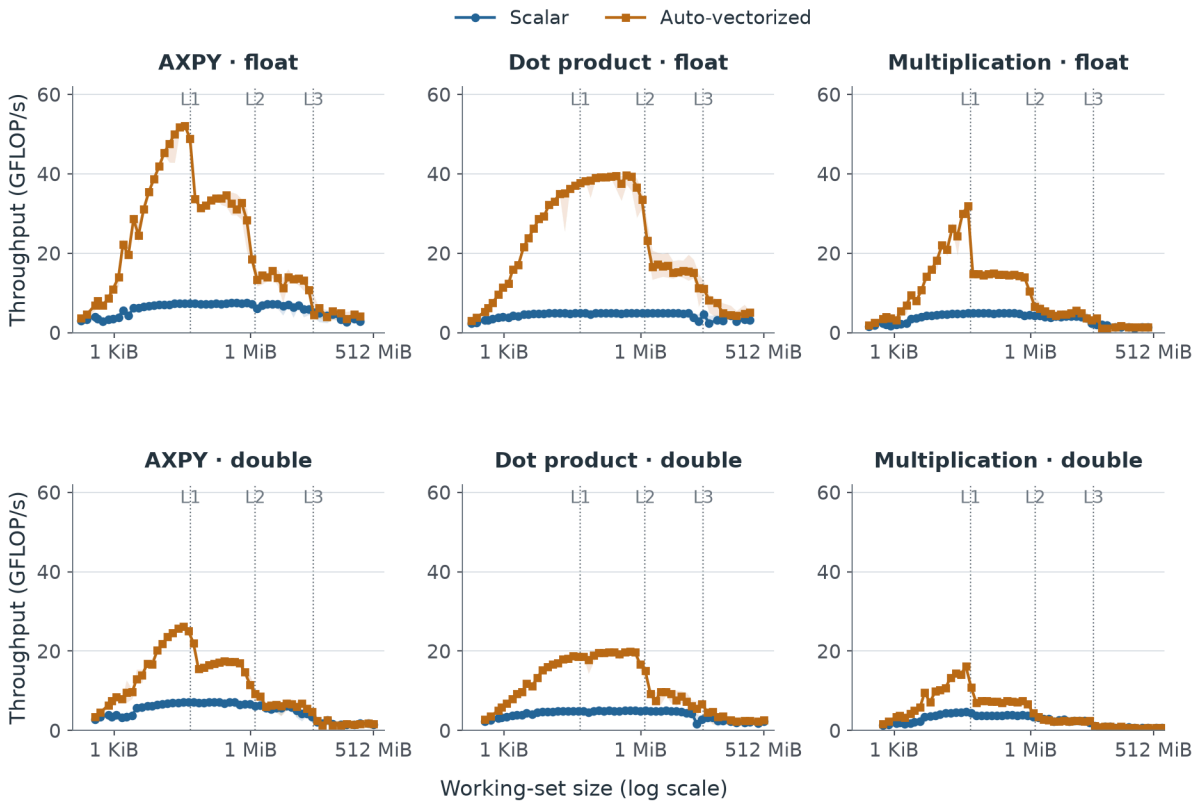


Fig. 2. Scalar and auto-vectorized throughput across working-set sizes. GFLOP/s measures billions of floating-point operations per second; higher is faster. Bands show the interquartile range.

Auto-vectorized throughput peaked at 52.13, 39.59, and 31.93 GFLOP/s for float AXPY, dot product, and multiplication, respectively (Figure 2). The double-precision peaks were 26.17, 19.84, and 16.16 GFLOP/s. Scalar peaks were 4.69–7.47 GFLOP/s.

Kernel/type	N = 29,000 Scalar (µs)	N = 29,000 Auto (µs)	N = 33,554,432 Scalar (ms)	N = 33,554,432 Auto (ms)
AXPY / float	8.026	1.712	24.204	16.183
AXPY / double	8.046	3.341	44.687	39.065
Dot/float	11.637	1.473	21.034	12.953
Dot / double	11.640	2.927	29.214	25.311
Multiply / float	5.903	2.007	25.032	23.827
Multiply / double	7.526	3.917	51.860	46.958

Table 1. Representative scalar and auto-vectorized runtimes. N is the number of elements; lower is faster.

Results compare array sizes that fit in L2 cache with sizes that require main-memory access.

Locality Sweep Across Cache Levels

SIMD gains were modest for tiny inputs and strongest for larger inputs that still fit in the L1 cache. Gains generally narrowed beyond the L1 cache as data movement became more restrictive. For the largest arrays, which exceeded L3 cache capacity, speedup was 1.07–1.47× across kernels and precisions, offering a relatively small inconsistent benefit.

Dot product sustains about 6.7–7.9× speedup for float and 3.5–4.0× for double in the L2-sized range, showing a stronger SIMD advantage than AXPY or element-wise multiplication. Dot product returns a single value rather than an array, avoiding repeated output writes. This reduces memory traffic and helps sustain the SIMD advantage through L2.

Figure 2 shows a similar throughput pattern, consistent with the speedup results in Figure 1. Auto-vectorized throughput is highest for cache-sized arrays, while the gap between scalar and vectorized performance narrows beyond L3.

Data-Type Comparison

Float generally achieved greater speedup and higher vectorized throughput (Figures 1 and 2). A 256-bit AVX instruction processes eight floats or four doubles, so floats allow twice as many values per instruction and require half as much memory traffic for the same number of elements.

Alignment and Tail Handling

We tested all three kernels at 12 array sizes. Each test started with a 64-byte-aligned array whose length filled complete AVX vectors. We then compared three changes: shifting the starting address by one element, adding seven elements to the array, and shifting the starting address and adding seven elements together. The shift introduced misalignment, while the extra elements left seven floats or three doubles to process after the last complete vector.

Figures 3 and 4 use the geometric mean to summarize the three kernels into a single speedup value, making the test conditions easier to compare. Each comparison uses a baseline measured in the same run.

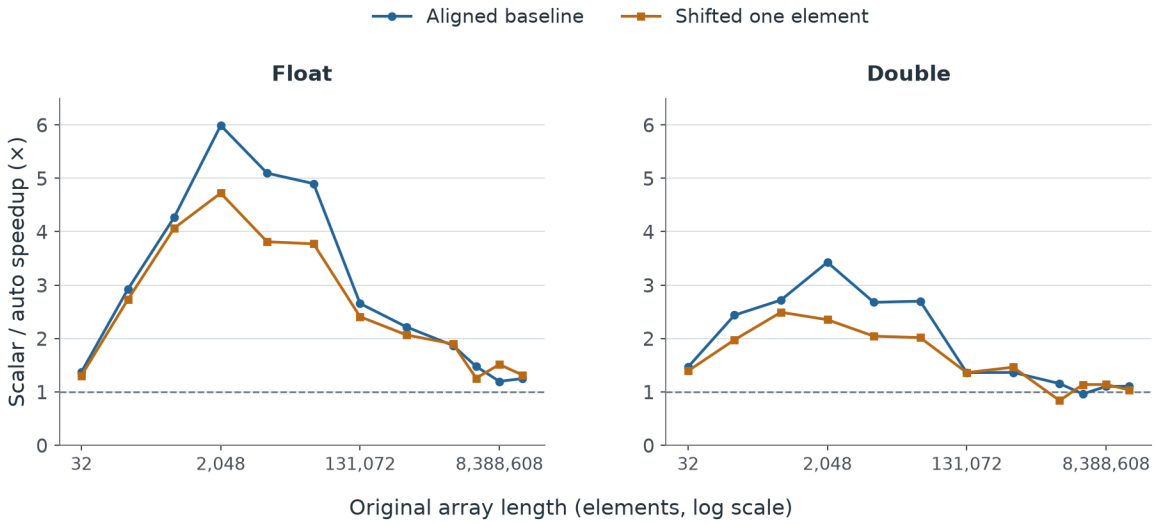


Fig. 3. Geometric-mean scalar-to-auto speedup across the three kernels for aligned and shifted arrays. Higher is faster; 1× means equal performance.

Shifted arrays generally achieved less SIMD speedup than aligned arrays for cache-sized inputs (Figure 3). For larger arrays, the curves moved closer together as memory traffic limited the benefit of vectorization.

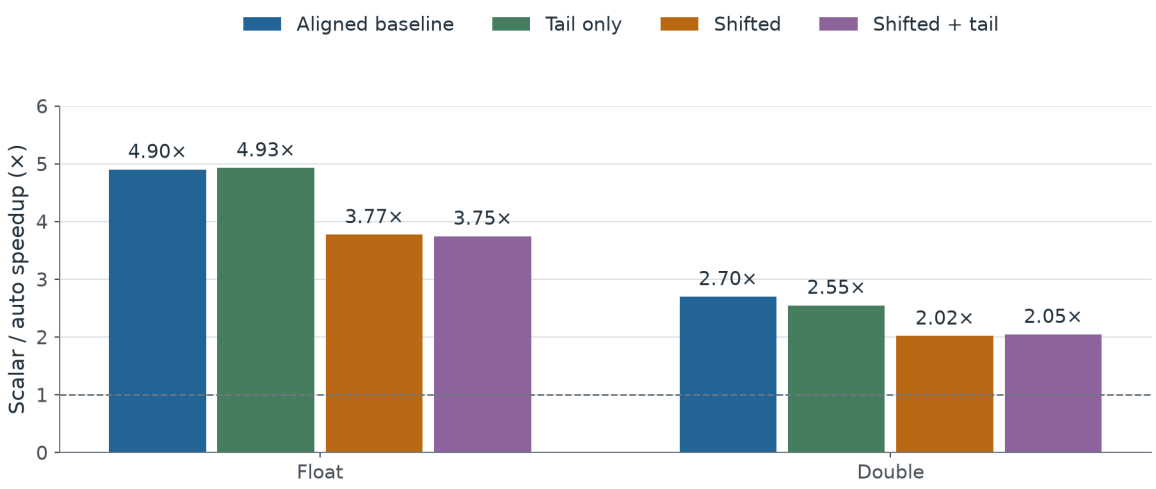


Fig. 4. Geometric-mean scalar-to-auto speedup across the three kernels at an original array length of 32,768 elements, comparing aligned, shifted, tail-only, and combined conditions.

Figure 4 separates the effects at an array length of 32,768 elements. This array length keeps all tests in the L2 cache regime, so alignment and tail effects can be compared without main-memory access dominating performance.

Shifting reduces average speedup from 4.90× to 3.77× for float and from 2.70× to 2.02× for double. Tail-only results remain closer to the baseline, at 4.93× and 2.55×. Applying both changes gives 3.75× and 2.05×, similar to shifting alone. At this size, misalignment therefore reduces the SIMD advantage more than the added tail.

Stride and SIMD Speedup

We tested AXPY at strides 1, 2, 4, and 8 using arrays that fit in L2 cache and larger arrays that required main-memory access. Figure 5 shows median scalar-to-auto speedup across 15 matched trials.

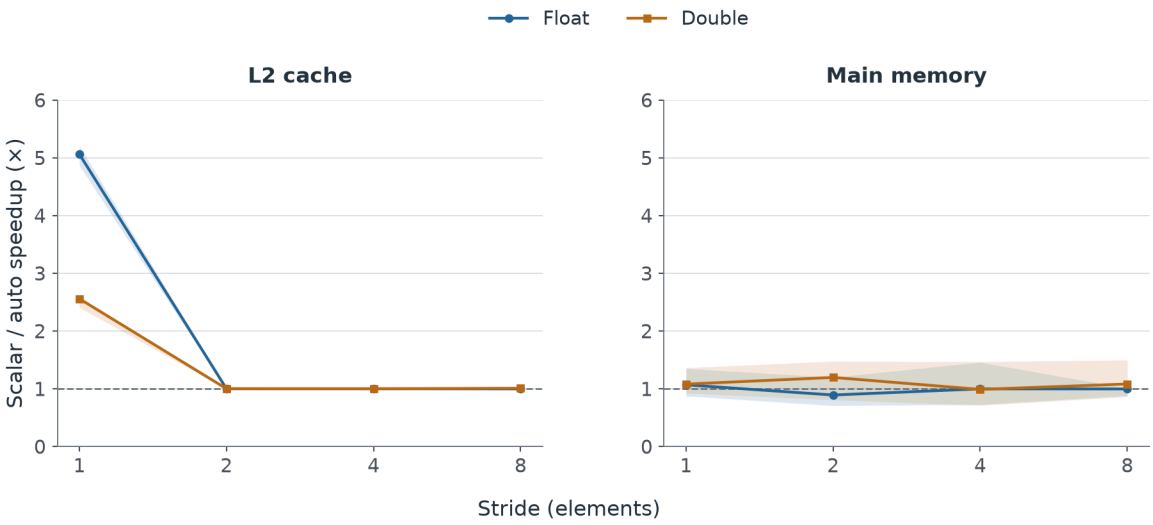


Fig. 5. AXPY scalar-to-auto speedup by stride; dashed line: equal performance (1×); bands: interquartile range.

For arrays that fit in L2 cache, SIMD provided a 5.06× speedup for float and 2.56× for double when accessing consecutive elements (stride 1). At strides 2–8, the speedup fell to about 1× because GCC did not vectorize these access patterns (Figure 5). For arrays requiring main-memory access, speedups remained near 1× across all strides (0.89–1.20×).

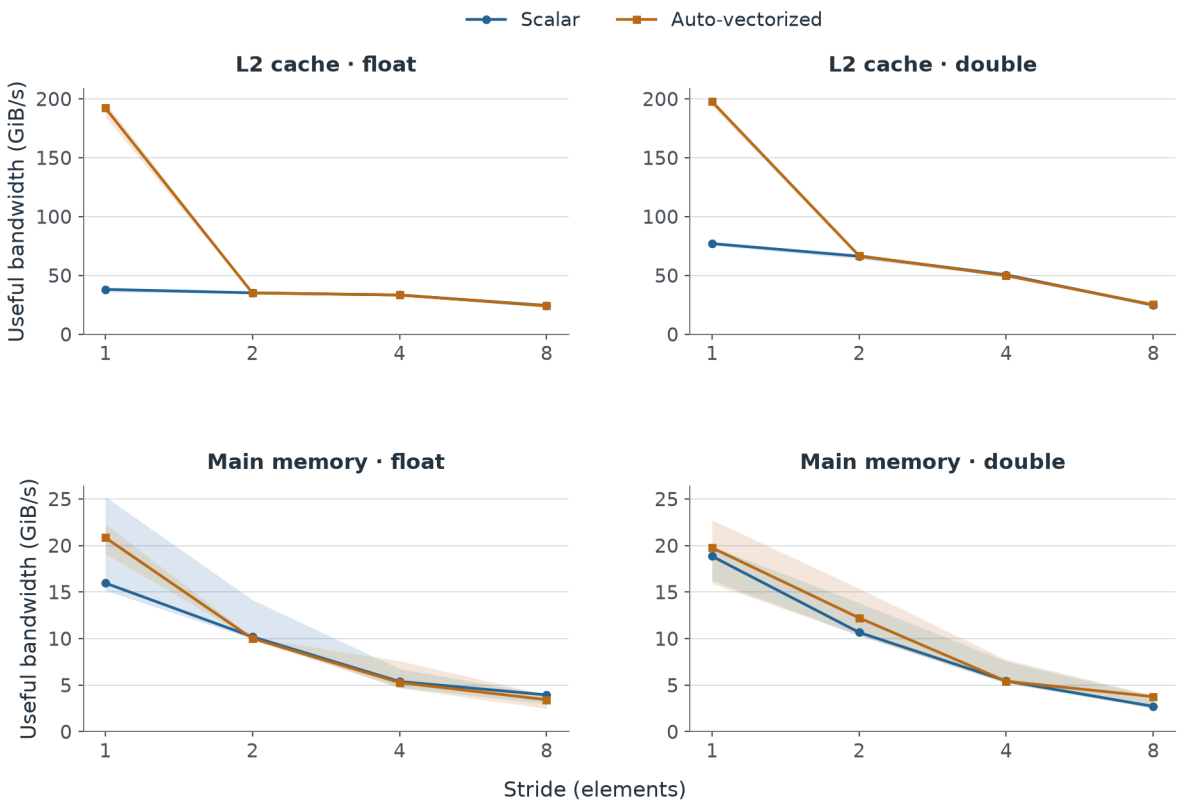


Fig. 6. Useful bandwidth by stride for arrays that fit in L2 cache (top) and arrays requiring main-memory access (bottom). Higher is faster. Bands show the interquartile range.

Figure 6 shows that increasing the stride reduced the amount of useful data processed per second. From stride 1 to stride 8, useful bandwidth fell from 20.84 to 3.42 GiB/s for float and from 19.75 to 3.76 GiB/s for double. Larger strides skip more values in each cache line, so more of the data fetched goes unused.

Vectorization Verification and Strict Dot Product

GCC's vectorization reports and generated instructions confirmed that all three kernels used 256-bit AVX vectors, the widest supported by this CPU. The scalar builds processed one value at a time. Fused multiply-add (FMA), which combines multiplication and addition into one instruction, was disabled in these comparisons.

We also tested dot product without -ffast-math, requiring additions to follow the original summation order. Speedup fell to 0.99–1.01×, compared with 8.06–8.56× for float and 2.99–4.07× for double when reordering was allowed (Figure 7). Although the strict version multiplied several values in parallel, it still added the products one at a time. This sequential addition removed nearly all of the SIMD speedup.

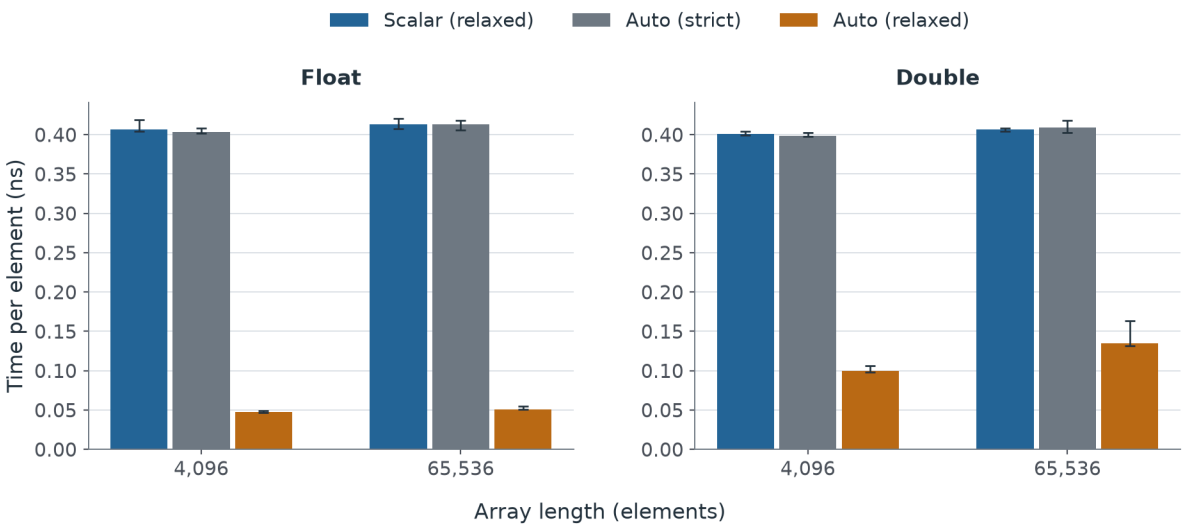


Fig. 7. Dot-product time per element with strict and relaxed floating-point summation. Lower is faster. Error bars show the interquartile range.

Native-Target Comparison

We compared a build with AVX enabled (-mavx) against one optimized for the test CPU (-march=native). Both used the same code and other compiler settings. We tested arrays that fit in L2 cache, fit in L3 cache, and required main-memory access. The table divides the AVX build's runtime by the CPU-specific build's runtime, so values above 1 indicate that CPU-specific compilation was faster.

Kernel/type	**32,768 L2-sized**	**1,048,576 LLC-sized**	**8,388,608 DRAM-sized**
AXPY / float	1.00× [0.99–1.01]	0.90× [0.82–1.18]	0.92× [0.61–1.26]
AXPY / double	1.00× [0.99–1.01]	1.11× [0.72–2.10]	1.03× [0.68–1.62]
Dot/float	1.00× [1.00–1.00]	1.00× [0.82–1.31]	0.94× [0.45–1.58]
Dot / double	1.00× [1.00–1.03]	1.08× [0.80–1.32]	0.84× [0.60–1.28]
Multiply / float	1.00× [0.98–1.02]	1.04× [0.76–1.28]	1.03× [0.75–1.45]
Multiply / double	1.02× [1.01–1.07]	1.01× [0.86–1.60]	1.04× [0.99–1.15]

Native targeting produced no consistent improvement. L2-sized median ratios were 1.00–1.02×; larger cases ranged from 0.84× to 1.11×, with most interquartile ranges including 1×. Both builds used the same vector width without FMA, helping explain their similar cache-sized performance.

Roofline Interpretation

We used a separate test with arrays larger than L3 cache to measure main-memory bandwidth. The test processed 17.12 GB/s of useful data, corresponding to an estimated 22.82 GB/s after accounting for additional memory traffic from writing the output. AXPY reads two float values and writes one, moving 12 bytes for every two calculations. Using this data requirement, the measured bandwidth predicts an AXPY throughput of about 3.80 GFLOP/s.

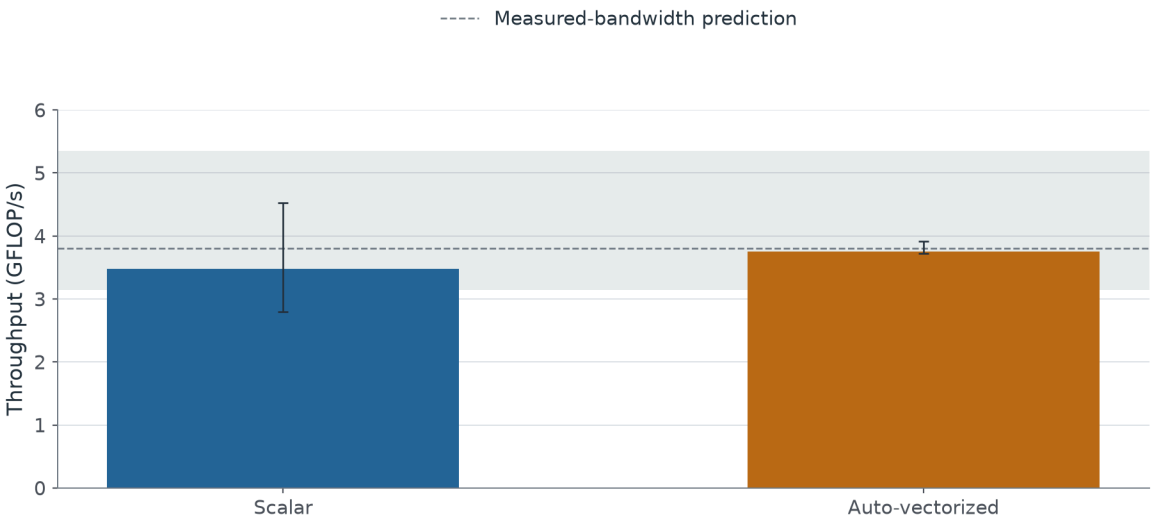


Fig. 8. Scalar and auto-vectorized SAXPY throughput compared with the estimated memory-bandwidth limit. Bars show medians, error bars show interquartile ranges, and shading shows the interquartile range of the estimated limit.

For arrays requiring main-memory access, vectorized SAXPY reached 3.76 GFLOP/s, compared with 3.48 GFLOP/s for scalar code (Figure 8). The vectorized result was close to the 3.80 GFLOP/s predicted from measured bandwidth, helping explain the limited 1.12× speedup. A separate FMA test kept its data in CPU registers and reached 147.63 GFLOP/s. This contrast supports the conclusion that data movement, not calculation speed, limited SAXPY.

Measured Cycles per Element

We measured the CPU cycles used by all three kernels across five array sizes, ranging from cache-sized arrays to arrays requiring main-memory access. Hardware counters recorded cycles throughout each kernel's execution.

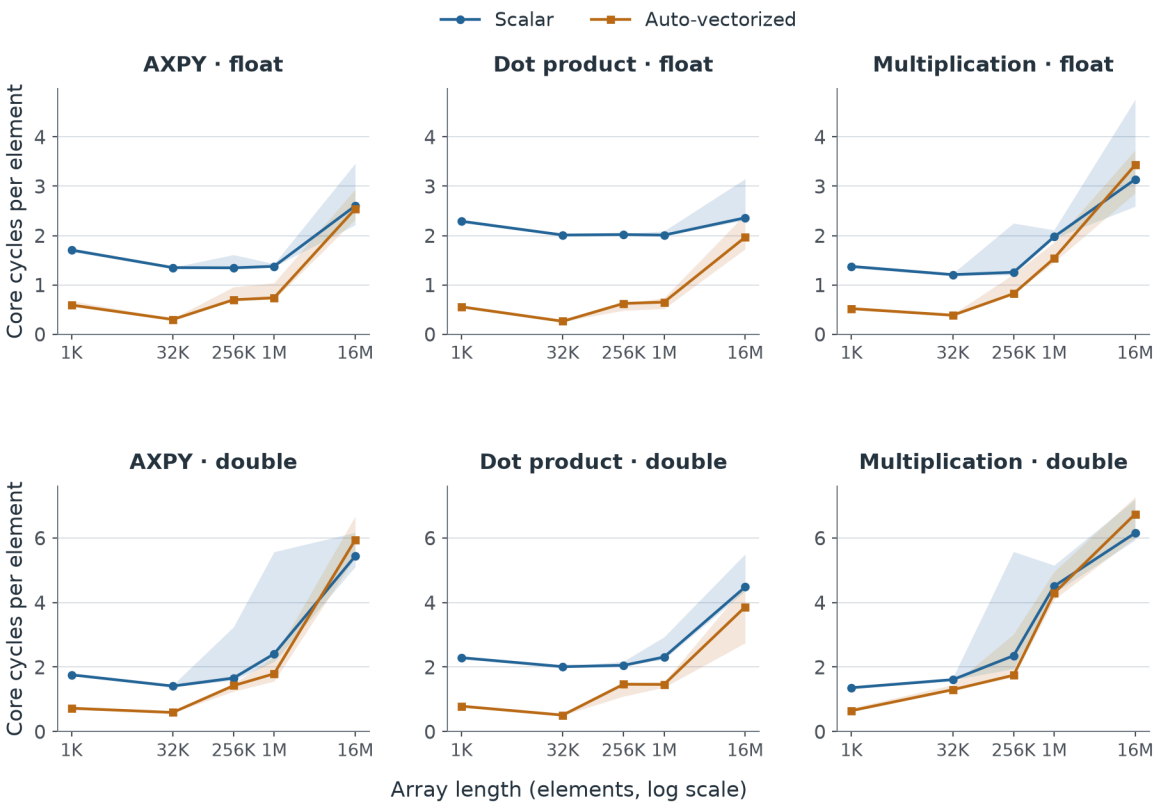


Fig. 9. Core cycles per element. Lower is better. Bands show the interquartile range. K = 1,024 and M = 1,048,576 elements.

For arrays that fit in L2 cache, vectorization reduced cycles per element from 1.35 to 0.30 for float AXPY, from 2.01 to 0.26 for dot product, and from 1.21 to 0.38 for multiplication (Figure 9). For the largest arrays, which required main-memory access, cycle counts increased, and the SIMD advantage narrowed.

Validation and Reproducibility

All reported correctness checks passed. We compared each result with a reference calculation and checked that the kernels left input-only arrays and memory outside the output arrays unchanged. These checks covered the main sweep, alignment and tail tests, stride and strict-dot tests, roofline measurements, cycle counts, and CPU-specific compilation. All 28 additional tests for memory errors and undefined behavior also passed.

The saved experiment records include source code, build and run scripts, raw measurements, processed results, plots, and reproduction instructions. We also retain compiler reports and generated instructions for checking vectorization.

Measurement Limits

1. Timing scope. We excluded setup, resets, and validation from timing, and warmups from cycle counts. Timer and call overhead remained, including 248 cycles for an empty cycle measurement.
2. Run-to-run variation. We did not control CPU frequency or background activity, and we retained all samples. Large-array results varied between runs, so the cause of the cycle-count crossover is uncertain. Interquartile ranges show the middle 50% of measurements, not confidence intervals.
3. Averages and cache ranges. Alignment averages can hide differences between kernels. Paired-trial speedups can differ from ratios of median runtimes. Cache boundaries are approximate, and cache misses were not measured.
4. Bandwidth and compute estimates. Useful bandwidth excludes unused data and extra memory traffic. The roofline prediction relies on a traffic model, rather than direct memory-traffic measurements. The FMA test provides a compute reference for kernels that did not use FMA.
5. Vectorization and rounding. Vector instructions do not guarantee a speedup, as the strict dot-product test shows. Reordering additions can also slightly change floating-point results.

AI Use

Codex assisted with C code, analysis scripts, figures, and drafting. We used reference comparisons, buffer checks, sanitizer tests, and disassembly inspection to evaluate the generated work.